

Práctica 3

Uso de interrupciones para leer valores de un botón

Grado en Ingeniería en Robótica Software

GSyC, Universidad Rey Juan Carlos



©2024 Julio Vega Pérez

©2025 Esther Aguado

Algunos derechos reservados.

Este trabajo se entrega bajo licencia CC-BY-SA 4.0.

1. Introducción

Las interrupciones son un mecanismo ampliamente utilizado en la programación embebida, ya que permiten que el microcontrolador responda a eventos externos en cualquier momento, incluso mientras se ejecuta el programa principal. Este enfoque resulta especialmente útil en situaciones donde se requiere una reacción inmediata, como al leer un encoder para no saltarnos ningún pulso o recibir datos a través del puerto serie por parte del usuario.

En esta práctica vamos a ver cómo manejar eventos asíncronos en Arduino mediante el uso de interrupciones. Para ello, utilizaremos un pulsador como dispositivo de entrada, y nuestro objetivo será detectar cuándo se ha presionado el botón. Además, abordaremos el problema del rebote del botón y cómo mitigarlo mediante técnicas de antirrebote tanto por software como por hardware.

2. Configuración de pines para el manejo de eventos

Hasta ahora hemos trabajado con los pines en su configuración de salida (*OUTPUT*), para manejar convenientemente un LED. Es una configuración sencilla, porque somos nosotros los que variamos a nuestro antojo la señal de salida del pin. Pero al leer de un pin hemos de tener en cuenta cómo es su señal de entrada, y es que esta oscilará entre HIGH y LOW si no está conectado a nada, por ejemplo cuando leemos de un sensor: típicamente, este dará una señal cuando *sense* algo. Este tipo de señal, como cualquier evento asíncrono, puede ocurrir en cualquier momento, y si no leemos el pin en el instante adecuado, no podremos distinguir la señal real del sensor de la oscilación natural del pin.

La solución es fijar un valor definido al pin mientras no se recibe una señal externa, de manera que podamos detectar con claridad cuándo cambia su estado debido a un evento. Esto se consigue mediante resistencias de *pull-up* y *pull-down*.

Una resistencia de *pull-up* se conecta entre el pin y la alimentación positiva de la placa (5V), de manera que el pin permanezca en nivel alto (HIGH) hasta que una señal externa, por ejemplo la pulsación de un botón, lo lleve a nivel bajo (LOW) al conectarlo a tierra.

Por el contrario, una resistencia de *pull-down* se conecta entre el pin y tierra (GND), asegurando que el pin se mantenga en nivel bajo (LOW) mientras no actúe ninguna señal, y que pase a nivel alto (HIGH) cuando se conecta a la alimentación positiva. Los valores recomendados para estas resistencias se sitúan normalmente entre 1 k Ω y 50 k Ω , siendo 10 k Ω la opción más habitual, ya que ofrece un equilibrio adecuado

entre consumo de corriente y tolerancia al ruido.

2.1. ¿Qué pasa si no usamos resistencias de pull-up o pull-down?

En este primer ejemplo, cuyo código tenemos en `boton_basico.ino` y el montaje en la figura 1, no funciona correctamente.

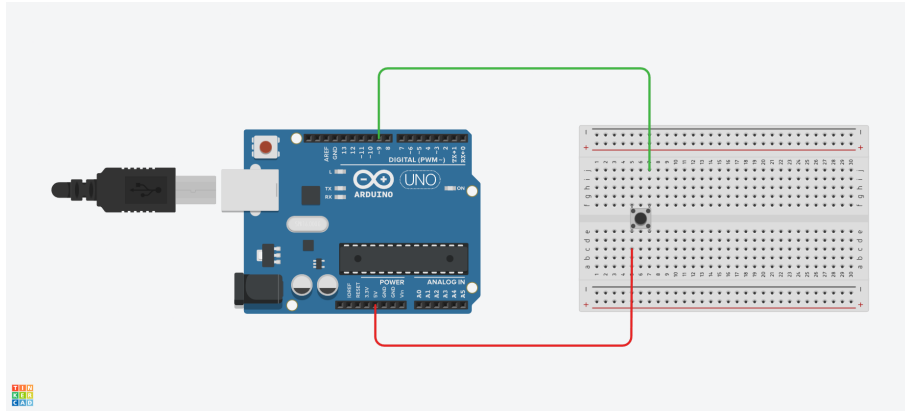


Figura 1: Esquema de conexión de pulsador, al no haber resistencias de pull-up ni pull-down, el pin queda flotante.

El código de `boton_simple.ino` lee un botón conectado al pin 9 y nos muestra el número de pulsación actual. Al no usar resistencias que fijen un valor, cuando el botón no está presionado, el pin queda flotante: puede leer HIGH o LOW de manera aleatoria.

```
1  const byte botonPin = 9;
2  bool pulsado = false;
3  int contador = 0;
4
5  void setup() {
6      pinMode(botonPin, INPUT);
7      Serial.begin(9600);
8  }
9
10 void loop() {
11     if (digitalRead(botonPin) == LOW) {
12         if (!pulsado) {
13             contador++;
14             Serial.println(contador);
15             pulsado = true;
16         }
17     }
18     else {
19         pulsado = false;
20     }
21 }
```

Los pines digitales de Arduino cuentan con una resistencia pull-up interna de aproximadamente $20\text{ k}\Omega$, que inicialmente está desconectada y solo se activa mediante software con `pinMode(pin, INPUT_PULLUP)`.

Para solucionar este problema, podríamos simplemente utilizar esta resistencia o colocarla físicamente en el circuito.

3. Ejemplo de manejo de eventos

A continuación veremos diferentes ejemplos prácticos sobre cómo manejar eventos. Para ver de forma más clara el sistema, en esta práctica vamos a montar la resistencia *pull-up* externa. Por tanto, vamos a añadir una resistencia de 10 k Ω entre el botón y la alimentación (5V), de manera que el pin permanezca en nivel alto (HIGH) hasta que se pulse el botón, que lo conectará a tierra y lo llevará a nivel bajo (LOW). El esquema de conexión seguido para estos ejemplos es el que aparece en la Figura 2.

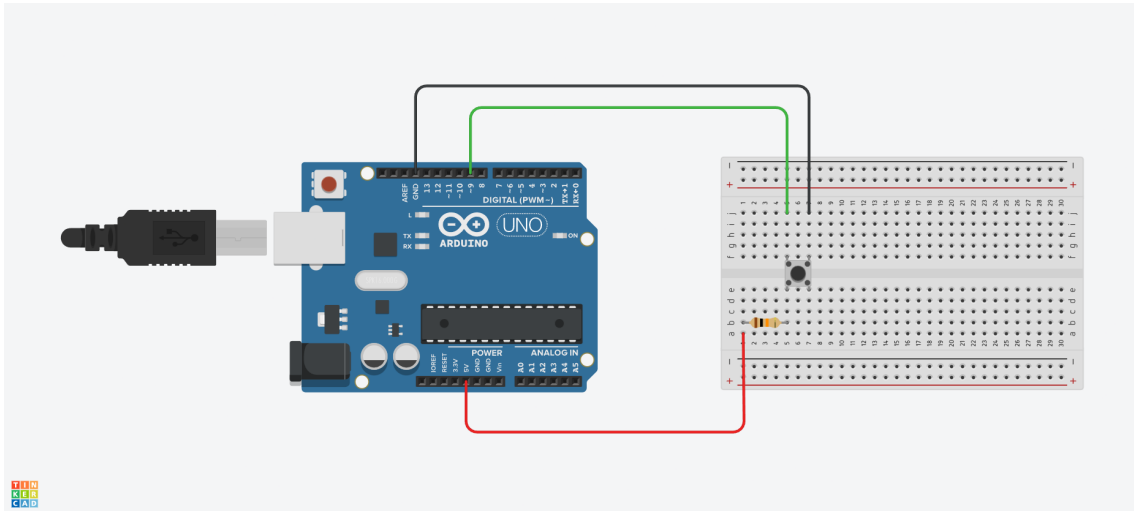


Figura 2: Esquema de conexión de pulsador empleado para estos ejemplos.

3.1. Interrupciones con `attachInterrupt`

Una vez tenemos el funcionamiento más básico, podemos mejorar la eficiencia del programa utilizando interrupciones para asegurarnos que los eventos sean atendidos. En condiciones normales, cuando el programa comienza su ejecución, sigue estrictamente las instrucciones para las que ha sido diseñado. Sin embargo, esta secuencia ordenada se ve alterada cuando incorporamos el manejo de eventos. La principal particularidad de los eventos es que ocurren de forma asíncrona; es decir, mientras el programa continúa realizando sus propias tareas, puede producirse un evento que debe ser atendido de inmediato.

Para facilitar la gestión de eventos asíncronos muchos microcontroladores incluyen soporte para interrupciones. Cuando ocurre un evento externo, como pulsar un botón o recibir una señal, el flujo normal del programa puede interrumpirse en cualquier momento para ejecutar una función especial llamada rutina de servicio de interrupción (ISR, por sus siglas en inglés). Esta ISR funciona como un *callback*: se ejecuta de inmediato cuando se activa la interrupción, sin que el programa principal tenga que comprobar constantemente si el evento ha ocurrido.

Para configurar una interrupción en Arduino¹, se utiliza la función `attachInterrupt` dentro de la función `setup()`. La sintaxis básica es la siguiente:

```
1 void setup() {  
2   // otras inicializaciones  
3   attachInterrupt(digitalPinToInterrupt(pin), ISR, mode);  
4 }
```

Esta función utiliza tres parámetros:

¹<https://docs.arduino.cc/language-reference/en/functions/external-interrupts/attachInterrupt/>

- `digitalPinToInterrupt(pin)`: Mapea el pin físico al número de interrupción correspondiente. Esto es necesario porque no todos los pines digitales pueden generar interrupciones. En Arduino Mega los pines que soportan interrupciones son: 2, 3, 18, 19, 20 y 21.
- `ISR`: Es el nombre de la función que se ejecutará cuando ocurra la interrupción. Esta función debe tener un tipo de retorno `void` y no debe aceptar parámetros.
- `mode`: Define el tipo de evento que activará la interrupción. Los modos disponibles son:
 - `RISING`: La interrupción se activa cuando la señal cambia de LOW to HIGH (activacion por flanco de subida).
 - `FALLING`: La interrupción se activa cuando la señal cambia de HIGH a LOW (activacion por flanco de bajada).
 - `LOW`: se activa mientras el pin permanezca en nivel bajo.
 - `CHANGE`: se activa cada vez que el pin cambia de valor (de alto a bajo o de bajo a alto).

El código completo se encuentra en el archivo `boton_interrupcion.ino`. En este caso, el botón está conectado al pin 2, que acepta interrupciones. Como la pulsación hace que el pin cambie de HIGH a LOW, configuramos la interrupción con el modo `FALLING` (si hubiéramos usado un pull-down el botón estaría a tierra sin pulsar por lo que deberíamos detectar la subida). La función `interrupcionBoton` es la que establece que el botón ha sido pulsado por lo que ya no es necesario comprobar continuamente el estado del pin como en el ejemplo anterior.

```

1  const byte botonPin = 2;
2  volatile bool pulsado = false;
3  int contador = 0;
4
5  void setup() {
6    Serial.begin(9600);
7    pinMode(botonPin, INPUT);
8    attachInterrupt(digitalPinToInterrupt(botonPin), interrupcionBoton, FALLING);
9  }
10
11 void loop() {
12   if (pulsado) {
13     contador++;
14     Serial.println(contador);
15     pulsado = false; // Reset estado del boton
16   }
17 }
18
19 // Funcion llamada por la interrupcion
20 void interrupcionBoton() {
21   pulsado = true;
22 }

```

4. Rebotes en el pulsador

En todos los ejemplos anteriores hemos obviado un problema común al trabajar con botones mecánicos: el rebote. Al pulsar el botón una sola vez, en realidad hay varias subidas y bajadas hasta que la señal se estabiliza.

Este fenómeno se conoce como *rebote* o *bouncing*, como se muestra en la Figura 3. Para evitar detectar un mismo evento varias veces, es necesario implementar un mecanismo de antirrebote, que puede realizarse mediante software, hardware o una combinación de ambos.

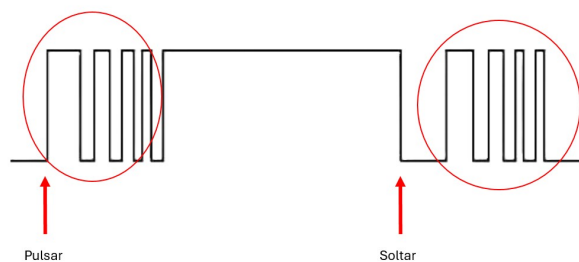


Figura 3: Ejemplo de rebote en un pulsador.

1. Antirrebote por software

Para mitigar el efecto del rebote mediante software, podemos comprobar el tiempo transcurrido entre disparos consecutivos de la interrupción. Si este tiempo es menor que un determinado umbral, simplemente se ignora la interrupción. En otras palabras, definimos una *zona muerta* durante la cual cualquier cambio en el estado del botón es descartado. Este método es sencillo de implementar, pero puede no ser el más eficiente, ya que introduce un pequeño retardo en la respuesta del sistema.

2. Antirrebote por hardware

Alternativamente, se puede implementar un filtro hardware que estabilice la señal del pulsador. Un condensador en serie a una resistencia forma un filtro RC, que suaviza los rebotes mecánicos: el condensador se carga y descarga gradualmente, evitando que las transiciones rápidas generadas por el rebote lleguen directamente al microcontrolador.

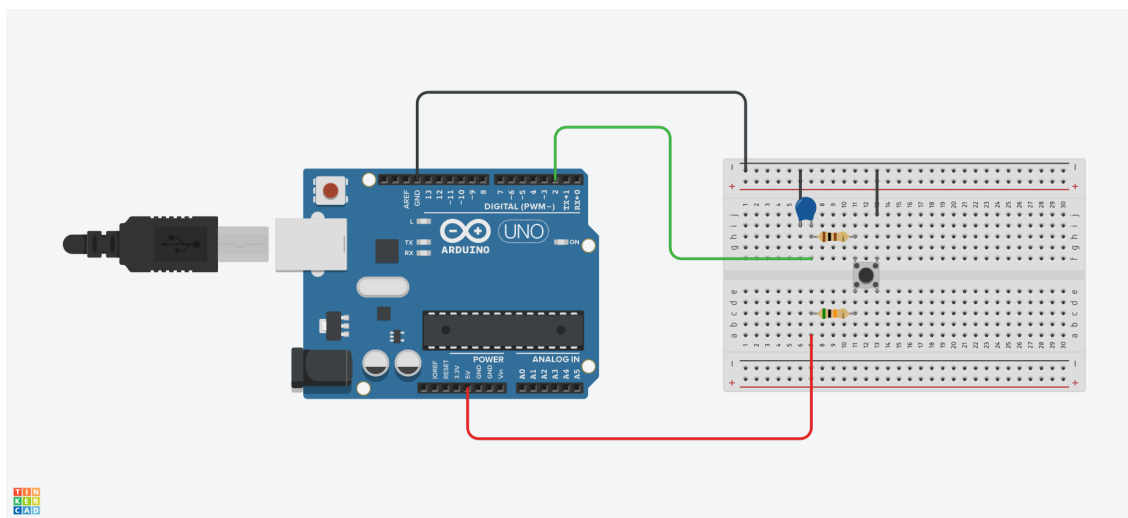


Figura 4: Ejemplo de circuito antirrebote por hardware.

La figura 4 muestra el esquema de conexión del circuito antirrebote por hardware. Como hasta ahora, la resistencia de $10\text{ k}\Omega$ actúa como un pull-up pasivo, manteniendo el pin en nivel alto (HIGH) cuando el botón no se encuentra presionado. El condensador de 100 nF colocado en serie forma el filtro RC. Además, añadimos una resistencia de $100\text{ }\Omega$ en serie con el pulsador para limitar la corriente durante la conmutación, protegiendo tanto al condensador como al microcontrolador.

Los valores de los componentes del filtro se eligen para obtener una constante de tiempo, (τ), adecuada que permita filtrar los rebotes sin introducir un retardo excesivo en la respuesta del sistema. Esta constante de tiempo determina la rapidez con la que se carga y descarga el condensador y por tanto, el tiempo que filtra la señal. El cálculo de la constante de tiempo τ se realiza mediante la fórmula:

$$\tau = R \cdot C = 10000 \cdot 100 \cdot 10^{-9} = 0,001s$$

Para nuestra resistencia de 10 k Ω y el condensador de 100 nF, obtenemos una constante de tiempo de 1 ms. Este valor de tiempo es suficiente para filtrar los rebotes típicos de un pulsador mecánico, que suelen durar entre 1 y 5 ms.

En la práctica, para mayor fiabilidad, se suele combinar ambos métodos. El antirrebote por hardware reduce significativamente el ruido en la señal, mientras que el antirrebote por software proporciona una capa adicional de filtrado para asegurar que los eventos se registren de manera precisa y consistente.

Ejercicios

Ejercicio 1

Aplica un filtro antirrebotes combinando métodos software y hardware sobre el método con interrupciones modificando `boton_interrupciones.ino`. Al fichero resultante llámalo `boton_interrupciones_antirrebotes.ino`

Ejercicio 2

Rediseña el esquema de conexión, así como la implementación de los tres códigos facilitados para dar soporte a dos botones que a su vez operan sobre dos LEDs (o un LED RGB) de forma independiente. Así, al pulsar uno de los botones, se encenderá un LED (y se apagará al soltar dicho botón); y, al pulsar el otro botón, lo mismo pero con el otro LED.

Recuerda que deberás ofrecer esa funcionalidad bajo las tres implementaciones que se han expuesto aquí. Si no tienes suficientes condensadores, no es necesario aplicar el antirrebote por hardware. Los códigos resultantes estarán en ficheros con estas denominaciones:

1. `doble_boton_simple.ino`
2. `doble_boton_interrupcion.ino`
3. `doble_boton_interrupcion_antirrebotes.ino`

Entrega de la práctica

La entrega se realizará en el repositorio individual de GitHub Classroom. Cada repositorio debe contener al menos el código producido así como un archivo `README.md` que incluya la memoria de la práctica con los siguientes elementos:

La memoria de prácticas debe contener los siguientes elementos:

- Nombre de los estudiantes.
- Objetivo de la práctica.
- Resumen del trabajo realizado.
- Particularidades: Problemas encontrados, funcionalidad extra, etc.

- Vídeo del funcionamiento del ejercicio.
- Esquemático del circuito (Fritzing, Tinkercad, etc.).